# GA7-220501096-AA1-EV01 INFORME TÉCNICO DE PLAN DE TRABAJO PARA CONSTRUCCIÓN DE SOFTWARE

**MIGUEL ANGEL SAGANOME LOPEZ**

**SERVICIO NACIONAL DE APRENDIZAJE SENA**

**CENTRO DE DESARROLLO AGROINDUSTRIAL Y EMPRESARIAL**

**TECNÓLOGO ANÁLISIS Y DESARROLLO DE SOFTWARE ADSO**

**BOGOTA D.C**

**2026**

# GA7-220501096-AA1-EV01 INFORME TÉCNICO DE PLAN DE TRABAJO PARA CONSTRUCCIÓN DE SOFTWARE

**MIGUEL ANGEL SAGANOME LOPEZ**

## INFORME TÉCNICO ESPECIFICANDO HERRAMIENTAS Y TECNOLOGÍAS DE VERSIONAMIENTO A UTILIZAR

**INSTRUCTOR**

**ESTEBAN ERNESTO MORALES CASTRO**

**INGENIERO DE SISTEMAS**

**SERVICIO NACIONAL DE APRENDIZAJE SENA**

**CENTRO DE DESARROLLO AGROINDUSTRIAL Y EMPRESARIAL**

**TECNÓLOGO ANÁLISIS Y DESARROLLO DE SOFTWARE ADSO**

**BOGOTA D.C**

**2026**

**Contenido**

INTRODUCCION..... 4

OBJETIVO..... 5

SELECCIÓN HERRAMIENTAS DE VERSIONAMIENTO..... 6

CONCLUSIONES..... 10

# INTRODUCCION

En el desarrollo de software moderno, el control de versiones es un componente fundamental para garantizar la organización, trazabilidad y mantenimiento eficiente del código fuente. En el contexto de un sistema destinado a la gestión de una escuela de fútbol que incluye funcionalidades como registro de estudiantes, control de asistencia, gestión de personas, entrenamientos, partidos, administrativos; es imprescindible implementar herramientas que permitan el trabajo colaborativo, la recuperación de versiones anteriores y el seguimiento de cambios.

Este informe técnico presenta la selección y justificación de herramientas de versionamiento local, destacando el uso de sistemas como Git, así como tecnologías complementarias que apoyan el flujo de trabajo del desarrollo.

# OBJETIVO

## Objetivo General

Definir y justificar las herramientas y tecnologías de versionamiento local que serán utilizadas en el desarrollo del sistema de gestión de una escuela de fútbol.

## Objetivos Específicos

- Identificar herramientas de control de versiones adecuadas para el proyecto.
- Establecer un flujo de trabajo eficiente para el manejo del código fuente.
- Garantizar la integridad, trazabilidad y colaboración en el desarrollo del software.
- Proponer buenas prácticas en el uso del versionamiento.

# SELECCIÓN HERRAMIENTAS DE VERSIONAMIENTO

## Sistema de Control de Versiones: Git.

Git es un sistema de control de versiones distribuido ampliamente utilizado en la industria del software. Permite a los desarrolladores trabajar de manera local, manteniendo un historial completo de cambios en el repositorio.

## Características principales.

- Control de versiones distribuido (no depende de conexión constante a servidor).
- Manejo eficiente de ramas (branching) y fusiones (merging).
- Historial completo de cambios.
- Alta velocidad y rendimiento.

Git es la herramienta ideal para este proyecto debido a su flexibilidad, robustez y amplia adopción. Permite que múltiples desarrolladores trabajen simultáneamente sin conflictos significativos, lo cual es esencial para el desarrollo del sistema de gestión.

## Interfaz Gráfica para Git: GitHub Desktop

GitHub Desktop es una herramienta que facilita el uso de Git mediante una interfaz gráfica amigable.

## Características.

- Visualización de cambios en archivos.
- Gestión sencilla de commits.
- Manejo de ramas sin necesidad de línea de comandos.
- Integración con repositorios remotos.

Es especialmente útil para desarrolladores con menor experiencia en línea de comandos, facilitando la adopción de buenas prácticas de versionamiento.

**Plataforma de Repositorios Remotos: GitHub**.

Aunque el enfoque principal es el versionamiento local, el uso de GitHub como complemento para respaldo y colaboración remota.

- Almacenamiento de repositorios en la nube.
- Control de acceso y colaboración.
- Integración con herramientas de integración continua.

Permite mantener copias de seguridad del repositorio local y facilita el trabajo en equipo, especialmente si el proyecto crece.

**Editor de Código Integrado: Visual Studio Code**.

Visual Studio Code es un editor de código que incluye integración nativa con Git.

- Control de versiones integrado.
- Extensiones para mejorar el flujo de trabajo.
- Interfaz intuitiva.
- Soporte para múltiples lenguajes.

Permite gestionar el versionamiento directamente desde el entorno de desarrollo, optimizando tiempos y procesos.

***Flujo de Trabajo Propuesto***

Para el desarrollo del sistema de gestión de la escuela de fútbol, se propone el siguiente flujo de trabajo basado en Git:

1. Inicialización del repositorio local
    - Uso de git init.
2. Creación de ramas
    - Rama principal: main
    - Ramas de desarrollo: feature/nombre-funcionalidad
3. Registro de cambios (commits)
    - Commits frecuentes y descriptivos.
4. Fusión de ramas
    - Uso de merge o pull request.

5. Respaldo remoto (opcional pero recomendado)

- Uso de GitHub para sincronización.

**Buenas Prácticas de Versionamiento**

- Realizar commits pequeños y frecuentes.
- Utilizar mensajes de commit claros y descriptivos.
- Evitar trabajar directamente sobre la rama principal.
- Documentar cambios importantes.
- Mantener el repositorio organizado.

# CONCLUSIONES

El uso de herramientas de versionamiento como Git es fundamental para el desarrollo exitoso del sistema de gestión de una escuela de fútbol. La implementación de estas tecnologías no solo mejora la organización del código, sino que también facilita la colaboración entre desarrolladores, reduce errores y permite una mejor trazabilidad del proyecto.

La combinación de Git con herramientas complementarias como GitHub Desktop, GitHub y Visual Studio Code proporciona un entorno robusto, eficiente y escalable para el desarrollo del software.